OS Orange Problem - U4

SRN: PES2UG24CS090
Name: Ashmita Chaki
Github link: https://github.com/ashyune/OS-U4-Orange

Screenshots

1A

```
❯ make test_objects
gcc -Wall -Wextra -O2 -c test_objects.c -o test_objects.o
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto

~/Desktop/Ash/OS-Orange/OS-U4-Orange main
❯ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B

```
❯ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
```

2A

```
❯ make test_tree
gcc -Wall -Wextra -O2 -c test_tree.c -o test_tree.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto

~/Desktop/Ash/OS-Orange/OS-U4-Orange main
❯ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

2B

```
~/Desktop/Ash/OS-Orange/OS-U4-Orange main
❯ set TREE_OBJ (find .pes/objects -type f | head -n 1)

~/Desktop/Ash/OS-Orange/OS-U4-Orange main
❯ echo $TREE_OBJ


~/Desktop/Ash/OS-Orange/OS-U4-Orange main
❯ xxd "$TREE_OBJ" | head -20
xxd: No such file or directory
```

3A

```
~/Desktop/Ash/OS-Orange/OS-U4-Orange main
❯ ./pes init
Initialized empty PES repository in .pes/

~/Desktop/Ash/OS-Orange/OS-U4-Orange main
❯ echo "hello" > file1.txt

~/Desktop/Ash/OS-Orange/OS-U4-Orange main*
❯ echo "world" > file2.txt

~/Desktop/Ash/OS-Orange/OS-U4-Orange main*
❯ ./pes add file1.txt file2.txt

~/Desktop/Ash/OS-Orange/OS-U4-Orange main*
❯ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  .gitignore
  untracked:  Makefile
  untracked:  README.md
  untracked:  commit.c
  untracked:  commit.h
  untracked:  index.c
  untracked:  index.h
  untracked:  object.c
  untracked:  pes.c
  untracked:  pes.h
  untracked:  test_objects.c
  untracked:  test_sequence.sh
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h
```

3B

```
~/Desktop/Ash/OS-Orange/OS-U4-Orange main*
❯ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776446078 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776446087 6 file2.txt
```

4A

```
~/Desktop/Ash/OS-Orange/OS-U4-Orange main*
❯ ./pes log
commit ec26489b53561a362c1007dc925083da70e23245e447d8d1456d3c0756a06278
Author: PES User <pes@localhost>
Date:   1776446266

    Add farewell

commit 87cef67c567cb96dfeb5c5df0339bc401e8452ee94094f860668856e96ffb6cf
Author: PES User <pes@localhost>
Date:   1776446236

    Add worldell

commit 5ff94d86f109d17cc6a5b3c45378ae4e26284e038220d82f1722064f6f3cf80e
Author: PES User <pes@localhost>
Date:   1776446212

    Initial commit
```

4B

```
~/Desktop/Ash/OS-Orange/OS-U4-Orange main*
❯ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/11/6367c406d598d725a24c146640e5199898b20b1929ace1b3d019d8249778d9
.pes/objects/5f/f94d86f109d17cc6a5b3c45378ae4e26284e038220d82f1722064f6f3cf80e
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/87/cef67c567cb96dfeb5c5df0339bc401e8452ee94094f860668856e96ffb6cf
.pes/objects/bd/9b769bb4d9910c31b4f0e99e4375fffea35c2055130c248ff131f78c626442
.pes/objects/d8/f28203071e10a3bde605928368bc5b51871287a867d8d10382967099dde814
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/ec/26489b53561a362c1007dc925083da70e23245e447d8d1456d3c0756a06278
.pes/refs/heads/main
```

4C

```
~/Desktop/Ash/OS-Orange/OS-U4-Orange main*
❯ cat .pes/refs/heads/main
ec26489b53561a362c1007dc925083da70e23245e447d8d1456d3c0756a06278

~/Desktop/Ash/OS-Orange/OS-U4-Orange main*
❯ cat .pes/HEAD
ref: refs/heads/main
```

Final

```
~/Desktop/Ash/OS-Orange/OS-U4-Orange main*
❯ make clean
rm -f pes test_objects test_tree object.o tree.o index.o commit.o pes.o test_objects.o test_tree.o
rm -rf .pes

~/Desktop/Ash/OS-Orange/OS-U4-Orange main*
❯ make test-integration
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -Wall -Wextra -O2 -c index.c -o index.o
gcc -Wall -Wextra -O2 -c commit.c -o commit.o
gcc -Wall -Wextra -O2 -c pes.c -o pes.o
gcc -o pes object.o tree.o index.o commit.o pes.o -lcrypto
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)
```

```
--- First Commit ---
Committed: 423d2d89d043... Initial commit

Log after first commit:
commit 423d2d89d04314edb88f153932cb23498d4db433fe5a3a9c313d197cbb725b14
Author: PES User <pes@localhost>
Date:   1776446376

    Initial commit


--- Second Commit ---
Committed: 302c4a8635ad... Update file.txt

--- Third Commit ---
Committed: 1bdbc2ea5af5... Add farewell

--- Full History ---
commit 1bdbc2ea5af5179e2de9eb3bbcf024cb2b6f94de13cb481256cd93ffe29580b6
Author: PES User <pes@localhost>
Date:   1776446376

    Add farewell

commit 302c4a8635ad32ddc1be0f0ab11aa40f56402c256747de1c21dd5eb7efd9c5dd
Author: PES User <pes@localhost>
Date:   1776446376

    Update file.txt

commit 423d2d89d04314edb88f153932cb23498d4db433fe5a3a9c313d197cbb725b14
Author: PES User <pes@localhost>
Date:   1776446376

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
1bdbc2ea5af5179e2de9eb3bbcf024cb2b6f94de13cb481256cd93ffe29580b6
```

```
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/1b/dbc2ea5af5179e2de9eb3bbcf024cb2b6f94de13cb481256cd93ffe29580b6
.pes/objects/30/2c4a8635ad32ddc1be0f0ab11aa40f56402c256747de1c21dd5eb7efd9c5dd
.pes/objects/42/3d2d89d04314edb88f153932cb23498d4db433fe5a3a9c313d197cbb725b14
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b8da6ff

=== All integration tests completed ===
```

Phase 5 & 6, Analysis Questions

Q5.1: Checkout implementation & complexity

Checkout updates both metadata and working directory.
Metadata:

- Resolve branch -> commit hash (from refs)
- Update HEAD to point to branch
- Update index to match commit tree

Working directory:

- Read commit -> tree -> recreate files/folders
- Remove files not in target branch
- Restore file permissions

Why complex:

- Must avoid overwriting uncommitted changes
- Handle file <-> directory conflicts
- Apply deletions/renames safely
- Keep HEAD, index, and working tree consistent (atomicity)

Q5.2: Detect dirty working directory

Compare per file:

1. HEAD vs Index -> staged changes
2. Index vs Working dir -> unstaged changes
3. HEAD vs Target branch -> what checkout will change

Conflict condition:

- File is locally modified (dirty) and
- Target branch also changes it

-> Refuse checkout to avoid data loss.

Q5.3: Detached HEAD

- HEAD points directly to a commit (not a branch)
- New commits are created but not attached to any branch
- These commits can be lost

Recovery:

- Create a branch at that commit hash
- Continue work from there
- (Git uses reflog; simple VCS requires manual tracking)

Q6.1: Finding unreachable objects

Use mark-and-sweep GC:

Mark:

- Start from all branch tips (+ HEAD)
- Traverse commits -> trees -> blobs
- Store reachable hashes in a hash set (O(1) lookup)

Sweep:

- Delete objects not in reachable set

Estimate:

- ~100,000 commits (not 50×100k due to shared history)
- Total objects: hundreds of thousands to a few million

Q6.2: GC vs commit race condition

Problem:

- Commit writes objects (blobs/trees) first
- Before updating branch ref, objects are “unreachable”
- GC may delete them
- Final commit points to missing objects -> corruption

How Git avoids this:

- Uses locking for refs
- Delays deletion (keeps recent unreachable objects)
- Uses reflogs and safe pruning rules